Exercise 4: Dynamic DAG with YAML Configuration

Objective

Refactor the DAG to dynamically generate tasks from a YAML configuration file, and apply best practices.

Prerequisites

- Exercise 3 completed

Key Concept: Dynamic Task Generation

Since a DAG file is just Python, you can use **loops** to generate tasks dynamically instead of writing each one by hand.

In previous exercises we hardcoded each pipeline (fetch public APIs, fetch universities). If we need to add a third source, we would have to copy-paste the same functions again. Instead, we can iterate over a configuration and generate tasks programmatically:

```
sources = {"public_apis": {"url": "..."}, "universities": {"url": "..."}}

with DAG(...) as dag:
    for source_name, config in sources.items():

        @task(task_id=f"fetch_{source_name}")
        def fetch(url, name, ds=None):
            ...

        @task(task_id=f"to_silver_{source_name}")
        def to_silver(s3_key, name, ds=None):
            ...

        key = fetch(config["url"], source_name)
        to_silver(key, source_name)
```

This creates **one pair of tasks per source**, all running in parallel. Adding a new source means adding a line to the config, not touching the DAG code.

Note: each task needs a **unique** `task_id`, which is why we use f-strings with `source_name`.

Step 1: Review the YAML Configuration

Open `dags/sources.yaml`:

```yaml
chuck_jokes:
    url: https://api.chucknorris.io/jokes/search?query=computer
    result_key: result
universities_spain:
    url: http://universities.hipolabs.com/search?country=Spain
```

The `result_key` field is optional: use it when the API wraps the data inside a key (e.g., Chuck Norris returns `{"total": N, "result": [...]}`). When omitted, the full response is stored as-is.

Step 2: Review the Dynamic DAG

Open `dags/exercise4_dynamic.py`:

```
import json
import os
from datetime import datetime
from pathlib import Path

import boto3
import pandas as pd
import requests
import s3fs
import yaml
from airflow import DAG
from airflow.decorators import task

BRONZE_BUCKET = "bronze"
SILVER_BUCKET = "silver"
S3_ENDPOINT = os.environ.get("AWS_ENDPOINT_URL")

sources_path = Path(__file__).parent / "sources.yaml"
with open(sources_path) as f:
    sources = yaml.safe_load(f)

with DAG(
    dag_id="exercise4_dynamic",
    start_date=datetime(2026, 1, 1),
    schedule="0 14 * * *",
    catchup=False,
    default_args={
        "retries": 1,
    },
) as dag:

    for source_name, config in sources.items():

        @task(task_id=f"fetch_{source_name}")
        def fetch_and_upload(url, name, result_key=None, ds=None):
            response = requests.get(url)
            data = response.json()
            records = data[result_key] if result_key else data

            s3 = boto3.client("s3", endpoint_url=S3_ENDPOINT)
            s3_key = f"{name}/_created_date={ds}/dump.json"

            s3.put_object(
                Bucket=BRONZE_BUCKET,
```

```
                Key=s3_key,
                Body=json.dumps(records),
            )

            return s3_key

        @task(task_id=f"to_silver_{source_name}")
        def to_silver(s3_key, name, ds=None):
            fs = s3fs.S3FileSystem(
                endpoint_url=S3_ENDPOINT,
                key="minioadmin",
                secret="minioadmin",
            )

            with fs.open(f"{BRONZE_BUCKET}/{s3_key}") as f:
                df = pd.read_json(f)

            silver_key = f"{SILVER_BUCKET}/{name}/_created_date=
{ds}/data.snappy.parquet"
            with fs.open(silver_key, "wb") as f:
                df.to_parquet(f, compression="snappy", index=False)

            return f"s3://{silver_key}"

        bronze_key = fetch_and_upload(config["url"], source_name,
config.get("result_key"))
        to_silver(bronze_key, source_name)
```

Step 3: Best Practices Applied

Compare this DAG with the previous exercises:

1. **No local filesystem:** We use `s3.put_object()` to write directly to S3 instead of saving locally first
2. **Dynamic configuration:** Adding a new source only requires editing the YAML file
3. **Idempotent:** Re-running overwrites the same S3 keys (same date partition)
4. **Uses `ds`:** Not `datetime.now()`

Step 4: Add a New Source

Add a third source to `dags/sources.yaml`:

```yaml
chuck_jokes:
    url: https://api.chucknorris.io/jokes/search?query=computer
    result_key: result
universities_spain:
    url: http://universities.hipolabs.com/search?country=Spain
cat_facts:
    url: https://catfact.ninja/facts?limit=50
    result_key: data
```

Wait ~30 seconds for Airflow to detect the change, then trigger the DAG.

Verify in MinIO

Open http://localhost:9001 and check the `bronze` bucket. You should see 3 folders:

```
bronze/
  chuck_jokes/_created_date=.../dump.json
  universities_spain/_created_date=.../dump.json
  cat_facts/_created_date=.../dump.json
```

The DAG should show **3 parallel pipelines** in the Graph view, one for each source in the YAML file.

Questions

1. Why did we merge fetch + upload into a single task?
2. What are the trade-offs of reading configuration from a YAML file vs hardcoding?
3. Is this DAG idempotent? What would happen if you trigger it twice for the same date?
4. How would you add error handling for APIs that are down?